Orca 2.0!
Classic Meets *Even More* Modern: a (Slightly) Pragmatic Learning-Based Congestion Control for the Internet

Krithi Sub, Mircea Tatulescu

December 2025

Contents

1 Introduction

The internet is built on the assumption that a large number of independently operated end hosts will share a small set of network resources. Links and router buffers along an end-to-end path have finite capacity, yet applications continually attempt to send more data at faster rates across many different paths. Without some mechanism to coordinate these competing demands, packets would simply accumulate in queues, delays would grow without bound, losses would spike, and the network would collapse. Congestion control is the part of the transport layer, most notably TCP, that attempts to prevent this outcome by adapting the sender's sending rate to the available capacity along the path.

A congestion control algorithm must balance several goals:

1. It should maintain low end-to-end latency for interactive applications while still supporting high throughput bulk transfers.

2. It should be reasonably fair, so that different flows sharing a bottleneck link receive comparable treatment.

3. It must remain stable in cases where faced with delayed and/or noisy feedback.

As a result, designing congestion control requires mapping these limited signals to good sending decisions across a wide range of network conditions.

Classic TCP algorithms have a simple design pattern: additive increase, multiplicative decrease (AIMD). The sender maintains a congestion window (`cwnd`) that controls how much unacknowledged data may be in the network. When acknowledgments arrive, and no congestion is detected, the sender slowly increases `cwnd` to probe for additional available bandwidth. When congestion is inferred, either from packet loss or explicit congestion notification, the sender reduces `cwnd`. Variants such as Cubic and BBR refine this basic idea with different window growth rules and signals; however, they remain largely hand-designed.

Congestion control algorithms are no longer able to keep up with the wide range of networks, from high-speed datacenter and backbone links to cellular and Wi-Fi networks. Buffers can be extremely small or large; round-trip times can vary by orders of magnitude. A single fixed set of heuristics is not optimal in all of these scenarios. Consequently, a congestion control variant that performs well in one environment can behave poorly in another. For example, if the `cwnd` is aggressively reduced in a network that has large buffers to maintain high throughput, the capacity remains unused.

This has motivated a line of work on learning-based congestion control, where mapping from observed network state to sending decisions is learned rather than fully handcrafted. These approaches use reinforcement learning to train a policy that responds to features such as recent throughput, delay, and loss, achieving a specific goal such as high throughput with minimal delay. In principle, a learned policy can adapt to different network conditions and can be retrained when network conditions or deployment environments change. In practice, a purely learning based design struggles to remain stable and safe under conditions that differ from the training data, as well as being capable of handling noisy signals.

Orca, introduced in the SIGCOMM 2020 paper "Classic Meets Modern: a Pragmatic Learning-Based Congestion Control for the Internet," proposes a hybrid design to navigate these trade-offs (Soheil Abbasloo, 2020). Instead of replacing TCP's congestion control logic entirely, Orca embeds a deep reinforcement learning (DRL) agent into the existing Linux TCP stack. The kernel continues to perform fine-grain, per-ACK window updates similar to a traditional congestion control algorithm; however, Orca defines a coarse-grain control loop that periodically aggregates statistics from each flow over a monitoring time period (`MTP`) and passes them to a DRL agent. The agent then adjusts higher level parameters that influence the underlying window dynamics.

The original Orca implementation targets a legacy software stack, including a Linux 4.13 kernel with a custom patch. This makes it challenging to reproduce and extend the system on current platforms. Moreover, Orca's design currently uses a fixed `MTP` (20 ms in the original experiments), which may not be ideal for all network conditions.

In this project, we revisit Orca and aim to improve it. Our goals are threefold:

1. We aim to reproduce the original Orca environment and experimental results, establishing a baseline that validates our understanding of the code and experimental setup.

2. We will port Orca's kernel modifications to a modern Linux kernel, preserving the hybrid congestion control design while making it easier to deploy and maintain on modern systems.

3. We plan to implement dynamic algorithm switching within the kernel to allow Orca to govern different underlying congestion control schemes (like BBR or Westwood) based on real-time network classification. We hope that this enables the Orca agent to enhance performance by modulating the most appropriate algorithm for the current condition (ex. wireless versus internet), rather than being limited to modifying only the TCP Cubic window it falls back on. In addition, we plan to extend Orca by introducing a dynamic `MTP` that adapts to the round-trip time (RTT) of the network. We argue that the static value of the original Orca design (a fixed 20 ms control loop) is suboptimal: too slow for very low latency environments (delayed actions), and unstable for high-latency paths (acting before feedback arrives). By scaling the `MTP` based on the measured RTT, we can synchronize the agent's decision frequency with the network's physical feedback loop.

2 Hypothesis

Our project has three main hypotheses about the behavior of Orca and our extensions under the planned experimental scenarios.

1. **H1 (Baseline reproducibility).** If we run Orca in an environment that closely matches the original setup (utilizing the published legacy patch, Mahimahi traces, and original test scripts), then our reproduction will achieve throughput and delay metrics that are statistically consistent with the published figures. This will establish a validated baseline for comparison.

2. **H2 (Impact of porting to a modern kernel).** If we port Orca's kernelspace logic to a modern Linux kernel, then the algorithm will initially perform slightly worse than the baseline implementation. This performance regression is expected because the pre-trained RL model relies on specific timing and network stack behaviors of the legacy stack, which differ significantly in modern kernels.

3. **H3 (Benefits of dynamic algorithm switching and `MTP`).** If we implement dynamic algorithm switching to select the optimal base congestion control scheme and simultaneously adapt the `MTP` to the flow's RTT, then the system will demonstrate improved robustness and efficiency across diverse network environments compared to the baseline run on a modern kernel. Specifically, we expect the algorithm switching to prevent performance degradation in non-standard environments where Cubic typically struggles, while the `MTP` will minimize control loop oscillations in high-latency scenarios.

3 Implementation

For all sections, we run QEMU using KVM acceleration, a custom kernel, a QCOW2 disk as a root filesystem, user-mode networking with port forwarding, and a serial console. For Linux 4.13, we use Ubuntu version 17.10 (Artful Aardvark). For Linux 6.16, we use Ubuntu version 24.04 (Noble Numbat). The full QEMU command is below:

```
qemu-system-x86_64 \
    -enable-kvm \ -m 16384 \ -smp 16 \ -cpu host \
    -hda ../{ubuntu-version}-vm-disk.qcow2 \
    -kernel ./arch/x86/boot/bzImage \
    -append "root=/dev/sda1␣rw␣console=ttyS0␣nokaslr␣pti=on␣ip=dhcp" \
    -nographic \ -netdev user,id=net0,hostfwd=tcp::2222-:22 \
    -device e1000,netdev=net0
```

This table summarizes our different implementations:

Implementation	Explanation
Baseline Cubic	The congestion control algorithm is TCP Cubic.
Baseline Orca (MLP)	The congestion control algorithm is Orca, with Cubic fallback.
MLP+Modified Reward	Orca, with a modified reward function.
LSTM+Modified Reward	Orca, with a different model (LSTM) and a modified reward function.
MLP+Modified Reward+Dynamics	The congestion control algorithm is dynamically chosen, coupled with MLP, a modified reward function, and a dynamically chosen monitoring time period.

3.1 Reproducing Orca on Legacy Linux Kernel 4.13

To establish a validated baseline, our first objective was to reproduce the original Orca results in their native environment. This involved compiling a custom Linux 4.13 kernel using the provided patch and setting up the corresponding userspace reinforcement learning agent. In this section, we will explain what we learned about the Orca kernelspace and userspace environment to provide context for future modifications.

3.1.1 Two-Level Architecture

Orca distinguishes itself from clean-slate learning approaches by adopting a hybrid control hierarchy that splits responsibilities between kernel and userspace.

- Fine-grained control (kernelspace): The underlying TCP stack (specifically TCP Cubic) continues to handle the immediate, per-ACK reactions. It maintains an "ACK clocking" mechanism to ensure that the sending rate naturally paces with network feedback. This prevents catastrophic instability seen in previous pure RL approaches, which attempt to micromanage every packet.
- Coarse-grained control (userspace): A deep reinforcement learning (DRL) agent runs as a userspace process. Every 20 ms (the `MTP`), the agent observes the aggregated network statistics and outputs a high-level guidance parameter to modify the underlying TCP behavior.

3.1.2 Kernel Patch

The core part of Orca's kernel implementation is the monitoring block introduced as a custom patch to the Linux TCP stack. We analyzed the C++ server code and the patch to understand its two primary functions:

- State aggregation: the modified kernel intercepts TCP signals (ACKs, SACKs, timeouts) to calculate real-time statistics that are not standard in the Linux TCP info structure. These include average throughput, loss rate, and queuing delay.
- Actuation via socket options: To bridge the gap between kernel and userspace, Orca introduces specific `setsockopt` and `getsockopt` hooks. For reading state, the userspace agent reads the accumulated metrics using a custom socket option, which are reset after every read to ensure the agent sees a fresh snapshot of its last 20 ms. For writing actions, the kernel accepts a congestion window `cwnd` modifier from the agent. Crucially, the kernel does not blindly apply this value. It uses the agent's output to scale the current `cwnd` calculated by TCP Cubic, effectively treating the DRL agent's output as a dynamic aggressiveness factor rather than a raw limit.

There are a few important files worth mentioning:

- `net/ipv4/tcp_deepcc.c`: a new networking module that implements the core logic for the monitoring block and action enforcer by aggregating raw signals into statistics required by the RL agent.
- `net/ipv4/tcp_input.c`: hooks added to critical TCP processing paths to measure RTT on every ACK, update throughput sample, and override the standard window update.
- `net/ipv4/tcp.c`: modified `do_tcp_{set,get}sockopt` to handle new socket options (for retriving statistics, the current congestion window, etc.).

- `include/linuc/tcp.h`: TCP socket struct modified to include instantiation of statistics and an orca_enable flag.
- `include/uapi/linux/inet_diag.h`: defines the communication protocol format by ensuring that when the userspace agent asks for the data, the kernel formats the bits properly.

3.1.3 Reinforcement Learning Model

There are three main parts to the Orca DRL agent (which is implemented in Tensorflow 1.14). A more detailed description of the RL model can be found in 3.3.

- State Space: The agent receives a vector of normalized network statistics from the kernel. To capture temporal dynamics, the original implementation stacks the current observation with previous observations, creating a history buffer (defaulting to ten steps) that allows the simple Multi-Layer Perceptron (MLP) to infer trends.
- Action Space: The agent outputs a continuous scalar value, α. This value maps to a multiplier for the congestion window: $cwnd_{new} = 2^{\alpha} \times cwnd_{cubic}$. When $\alpha = 0.0$, the action taken is simply to behave exactly like Cubic, while positive or negative values allow the agent to aggressively ramp up or throttle down based on congestion signals.
- Reward Function: The original reward function is based on a modified "power" metric (throughput/delay), penalizing high latency and packet loss to encourage an operating point that maximizes bandwidth usage while minimizing buffer bloat.

3.2 Kernelspace Modifications

3.2.1 Supporting Orca (DeepCC)

Orca is integrated into the kernel as the `deepcc` module, which acts as a learned congestion controller layered on top of the standard TCP congestion control framework. Rather than replacing the entire TCP stack, Orca only overrides the congestion window update, leaving loss recovery, retransmissions, and pacing to the existing kernel code. The kernel adds a global run-time knob, `net.ipv4.tcp_deepcc_enable`, and a per–flow flag in `struct tcp_sock` that together decide whether `deepcc` is active for a given connection. When either flag is set, the main congestion-control path (`tcp_cong_control()`) invokes `deepcc_update_cwnd()` on each ACK, allowing the RL agent to adjust the congestion window based on its learned policy. To support the existing Orca user-space code, the following configurations need to be made to the kernel in Table 7.

To supply Orca with a view of the network, the kernel-side hooks also maintain per-flow statistics such as delivered bytes, RTT samples, retransmission counters, and ECN markings. These values are exported to userspace via the Orca runtime, where they form the state vector for the RL model. All of this logic can be disabled at run time, so the kernel behaves like an ordinary TCP stack using a fixed congestion control algorithm (e.g., Cubic or BBR). This makes it possible to compare (i) baseline TCP, (ii) Orca alone, and (iii) Orca plus the meta-controller described next, and (iv) our new models plus the meta-controller under identical traffic traces.

3.2.2 Algorithm Switching Support

On top of the Orca-enabled kernel, we implemented an in-kernel meta-controller that can automatically switch between multiple TCP congestion control algorithms based on observed path characteristics. We first introduced an `enum tcp_meta_algo`, assigning stable identifiers to the algorithms we want to consider (Reno, Cubic, BBR, DCTCP, Westwood, Veno, Illinois, and several others). A new per-flow structure, `struct tcp_meta_state`, embedded in `struct tcp_sock`, tracks the currently active algorithm along with smoothed RTT, queueing delay, ECN-marking ratio, loss ratio, and several counters used to decide when a switch is warranted. The fields in this structure and their per-socket memory cost are summarized in Table 8.

The meta-controller classifies each TCP flow into a coarse "environment" using simple threshold rules over base RTT, queueing delay, ECN, and loss. For example, short-RTT paths with substantial ECN marking are labeled DATACENTER_ECN, long-RTT paths with low loss become HIGH_BDP, and highly lossy but low-queueing-delay paths resemble WIRELESS. The full set of environments and their classification rules are summarized in

Table 9.

Given the inferred environment, the function `tcp_meta_choose_algo()` selects the most appropriate congestion control algorithm (Figure 10). For instance, Westwood or Veno is preferred on lossy wireless-like links, BBR or Illinois on high-BDP paths, and BBR or Cubic for generic Internet conditions, depending on the measured queueing delay and loss level. The main control loop, `tcp_meta_controller()`, runs once per ACK at the end of `tcp_cong_control()`. It updates exponentially weighted moving averages for delay, ECN, and loss, maintains a "badness" counter when persistent queueing is observed, enforces a minimum time between switches, and, when necessary, calls `tcp_set_congestion_control()` to change the active algorithm at run time. Each successful or failed switch is logged for later analysis. We added the **TCP_CONG_NON_RESTRICTED** flag to all available TCP algorithm structs to allow us access to all algorithms

In unmodified Linux, each newly initialized TCP socket inherits a single system-wide default congestion control algorithm, chosen at boot time via the `net.ipv4.tcp_congestion_control` sysctl, and this choice remains fixed for the lifetime of the connection unless the application, or privileged userspace agent, explicitly overrides it with `setsockopt(TCP_CONGESTION)`. While it is true that the operating system knows whether a connection is wired or wireless, the kernel does not currently perform any automatic algorithm selection based on observed characteristic paths or interfaces. It is purely decided by the configuration or the application. In our design, the controller runs entirely inside the TCP stack rather than relying on repeated userspace calls. This avoids per-decision context-switch overhead, requires no application changes, and allows algorithm switches to be driven directly by transport metrics at ACK time.

3.3 Userspace Modifications

3.3.1 Reinforcement Learning Modifications

We also focused on optimizing the DRL agent's decision-making process, and we identified two key areas for improvement: the objective function (reward) and the neural network architecture.

Reward Function

The reward function is the signal that guides the agent toward desirable behavior. The baseline Orca implementation uses a linear utility function derived from the power metric described earlier. It calculates reward as:

$$R = (\alpha \cdot \texttt{Throughput} - \beta \cdot \texttt{Loss} \times \texttt{DelayFactor})$$

The `DelayFactor` is a step function that drops drastically if latency exceeds a threshold. While effective at maximizing peak throughput, we observed that the multiplicative nature of the delay penalty introduced instability. A sudden jitter in RTT could cause the reward to collapse to zero, causing the agent to over-correct and inducing the high variance seen in the baseline Orca delays. To improve stability, we replaced this linear function with a logarithmic utility function instead:

$$R = (\alpha \cdot \log(\texttt{Throughput}) - \beta \cdot \texttt{Delay} - \gamma \cdot \texttt{Loss}$$

The logarithmic term naturally enforces proportional fairness: the marginal utility of gaining 1 Mbps decreases as throughput rises, encouraging the agent to focus on stability once bandwidth is saturated. Furthermore, making the delay penalty additive (rather than multiplicative) ensures the gradient remains smooth even during latency spikes, preventing the variance observed in the baseline. We tuned the weights ($\alpha = 20, \beta = 50$ to balance aggressiveness with safety by prioritizing bandwidth utilization and strictly penalizing bufferbloat.

Architecture Exploration

We found that the agent's ability to interpret network states depends heavily on its neural architecture. Orca uses a simple MLP with two hidden layers. To handle time-series data (like rising queues), it relies on a brute-force approach, where the input state is a flattened vector containing the current observation concatenated with the previous k observations. While computationally efficient, this treats the history as a static image, making it difficult for the agent to distinguish between distinct temporal patterns (ex. delay is 50 ms and rising versus delay is 50 ms and falling).

We hypothesized that a recurrent neural network would better capture these temporal dynamics, and modified

the agent's `Actor` class to replace the dense layers with a Long Short-Term Memory (LSTM) cell. We reshaped the flattened input vector back into a time-series sequence `[Batch, Steps, Features]` and passed it through a dynamic RNN layer. The network maintains an internal hidden state, h_t, carrying context from previous steps implicitly rather than relying on a fixed input buffer. The intent was to enable the agent to "remember" long-term trends in network jitter that exceed the short input buffer of the MLP. As will be detailed in future sections, this change introduced significant computational overhead that ultimately led us to revert to using the MLP for the final hybrid system.

3.3.2 Networking Modifications

Finally, we addressed the limitations of the fixed control loop and the single-algorithm dependency by introducing dynamic behavior at the userspace level.

Dynamic Monitoring Time Period

The baseline Orca system enforced a rigid control frequency. Regardless of network conditions, the C++ server and Python agent exchanged state every 20 ms. While this makes the implementation simpler, it also creates a critical desynchronization between the agent's decision loop and the network's feedback loop (RTT). In high-latency paths (RTT >> 20 ms) the agent acts faster than the network allows. For example, on a 100 ms path, the agent executes five decision steps before the feedback from the first action reaches the sender. This causes the agent to over-correct and increase throughput, filling the buffer before it receives the first signal of congestion.

Our solution was to implement a heuristic dynamic `MTP`. We modified the userspace agent to calculate the optimal `MTP` for the next step based on the current RTT observation. The relationship is defined as: $\texttt{MTP}_{next} = RTT_{current}$. This ensures the agent waits for exactly one full feedback cycle to allow the effects of its previous action to materialize before making a new decision. We clamped this value to a safety range of [20 ms, 100 ms] to prevent CPU saturation at excessively low RTTs and inactivity at extremely high RTTs.

We extended the interprocess communication protocol in `rl-module/envwrapper.py` to compute this dynamic interval and append it to the action vector sent to the kernel. On the server side, we removed the hardcoded 20 ms timer override and implemented logic to parse and apply this variable wake-up timer dynamically and allow the RL agent to modulate its own temporal resolution.

Algorithm Switching Support

After the kernel was modified to support dynamic switching between underlying congestion control algorithms based on real-time environmental classification, userspace modifications also had to be made. When it is required to fallback from Orca to TCP Cubic, the intended behavior is that the userspace agent should no longer be in control. However, when the kernel autonomously switches the congestion control algorithm to an alternative like Westwood, the agent would continue to receive network statistics and calculate reward as if it were still in control. During training, this would cause the agent to update its policy based on rewards (ex. high throughput achieved by Westwood) that resulted in actions it did not take. To solve this, we implemented a handshake mechanism to make the agent aware of its active status. We modified the server to query the kernel via `getsockopt` for the currently active congestion control algorithm before every state report and appended a binary flag (`is_orca_active` to the state vector sent to userspace: 1.0 if the underlying algorithm is Orca-controlled Cubic, and 0.0 otherwise. In the Python agent (`rl-module/d5.py`), we updated the training loop to parse this flag and enforce a passive mode when Orca is not active. The agent will continue to observe the network state to maintain history context, but explicitly skips the training step (does not store the experience in the replay buffer or update weights). This ensures the policy learns only from actions it directly influenced.

4 Results and Evaluation

4.1 Traces

To evaluate the performance of our congestion control implementations in a reproducible manner, we utilized Mahimahi, a network emulation toolkit that allows for accurate and shell-based emulation of link conditions (Ravinet, [n. d.]). Mahimahi functions by creating a nested shell environment where all network traffic sent by

processes inside the shell is routed through synthetic uplink and downlink queues. It uses packet-delivery traces (text files containing a list of timestamps) to determine exactly when a packet is allowed to leave the queue and traverse the link.

In our architecture, Mahimahi is the physical layer enforcer between the Orca server (sender) and the client (receiver). The trace files define the varying bandwidth capacity over time. If a trace file has timestamps 1 ms apart, it simulates a link that can transmit one packet every millisecond. The modified Linux kernel (sender side) remains unaware of the emulator; it simply sees a network interface. When packets are held back by Mahimahi to simulate limited bandwidth or congestion, the kernel's TCP stack registers this as increased round-trip time or packet loss. The RL agent then observes these kernel-level statistics. We used a few different trace configurations to test different aspects of the congestion control algorithms:

1. Step Trace: This trace simulates a highly dynamic network where the available bandwidth changes dramatically at fixed intervals. The capacity fluctuates between 4 Mbps, 12 Mbps, and 24 Mbps every 10 seconds. This test stress-tests the algorithm's adaptability and convergence speed because the "steps" force the agents to react quickly: when bandwidth drops, it must drain the queue immediately to prevent latency spikes; when it rises, it must ramp up quickly to utilize the new capacity. This is similar to variable network environments like cellular environments with fading signals or shared medium networks, where contention levels shift abruptly. Note that the total link capacity (area under the curve) for this trace over 60 seconds is 800 Mb.

2. Wired48 Trace: This trace simulates a stable, high-bandwidth broadband connection. The trace file permits packet delivery at a constant rate, resulting in a fixed capacity of approximately 48 Mbps. This test benchmarks steady-state performance. In this scenario, a good algorithm should maximize throughput (using nearly 100% of the 48 Mbps) while maintaining a low, stable latency. This is similar to reliable fibers, or Ethernet links with minimal jitter. Note that the total link capacity (area under the curve) for this trace over 60 seconds is 2880 Mb.

3. ATT-LTE-driving-2016: This trace simulates a real AT&T LTE connection experienced by a user in a moving car. An LTE connection tests fluctuating throughput, occasional deep fades, short outages, and variable delays that come with mobility, changing signal quality, and scheduling in a commercial network. Whereas the step and wired48 traces test controlled responsiveness and steady-state behavior, the cellular trace tests whether the algorithm achieves good throughput and minimizes delays in unpredictable conditions.

To run these experiments, we utilize the emulation script provided, which sets up the Mahimahi environment, configures kernel parameters, and launches the agent. (Soheil-Ab, [n. d.]). There are several critical parameters worth mentioning:

- Link Delay: the one-way propagation delay, typically set to 10 ms (so the minimum RTT is 20 ms).
- Bandwidth: derived from the specific trace (ex. 48 Mbps for wired48).
- Queue Size: the size of the bottleneck buffer managed by Mahimahi. If the send transmits faster than the trace allows, packets accumulate here.

Queue size is calculated differently between the step and wired48 traces. For the Step trace, to induce bufferbloat, the queue size is hardcoded to 1000 packets. A 1000-packet buffer is excessively large for the lower bandwidth levels of the step trace (at 6 Mbps, 1000 packets represent several seconds of delay). This configuration exposes the fundamental weakness of loss-based algorithms like TCP Cubic. Since the buffer is too deep to overflow easily, Cubic will continue to increase its window, filling the queue and causing latency to spike. This setup is intended to prove that the Orca agent can successfully control delay using its reward function rather than relying on packet loss to signal congestion. For wired48, the script calculates the bandwidth-delay product (BDP) using $\texttt{BDP} = \texttt{Bandwidth} \times \texttt{MinRTT}$, and the queue size is $2 \times \texttt{BDP}$. This is a standard network provisioning rule to absorb small bursts without dropping packets.

In addition, when measuring sending rate (traffic egress) across time, there are two main statistics we look at: link utilization ($\frac{\texttt{Total Data Sent (AUC of Sending Rate)}}{\texttt{Total Link Capacity (AUC of Cap)}}$) and relative jitter ($\frac{|Rate_t - Rate_{t-1}|}{Rate_{t-1}}$). Links can be underutilized or overutilized, a good link utilization percentage is slightly below the maximum link capacity. The jitter is a measure of oscillation amplitudes (sending rate stability). For the step trace, capacity jumps massively every 10 seconds, so we use a windowed approach instead of an overall average: the 60 second experiment is spliced into 10 second windows that align with the trace step and we take the average of all percentage changes.

4.2 Performance Across Kernels

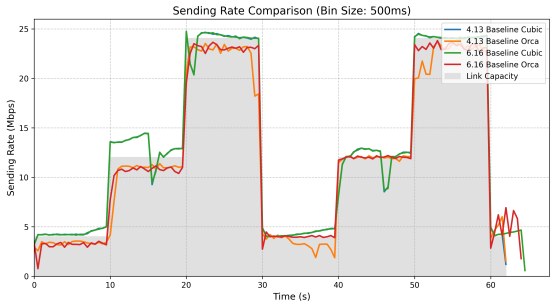

Figure 1: Sending Rate vs. Time: Step Trace

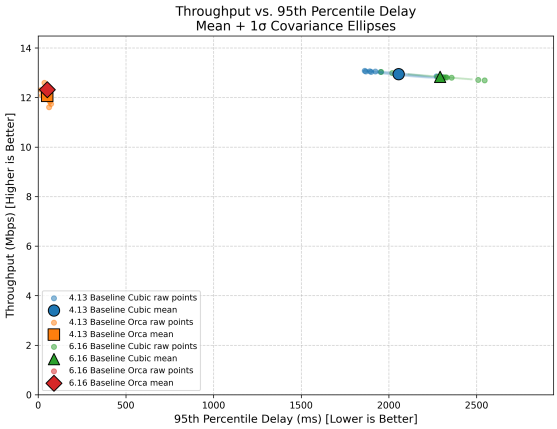

Figure 2: Throughput vs. Delay: Step Trace

Implementation (avgs over 10 runs)	**Link Util. (%)**	**Windowed Jitter (%)**
4.13 Baseline Cubic	101.990	3.169
4.13 Baseline Orca	92.510	5.810
6.16 Baseline Cubic	101.980	3.219
6.16 Baseline Orca	94.590	7.113

Table 1: Sending Rate Statistics: Step Trace

Implementation (avgs over 10 runs)	**Throughput (Mbps)**	**Delay (ms)**
4.13 Baseline Cubic	mean: 12.942 var: 0.019	mean: 2054.800 var: 36970.000
4.13 Baseline Orca	mean: 12.062 var: 0.078	mean: 51.500 var: 35.890
6.16 Baseline Cubic	mean: 12.834 var: 0.010	mean: 2292.100 var: 30640
6.16 Baseline Orca	mean: 12.313 var: 0.0153	mean: 52.400 var: 20.640

Table 2: Mean and Variance: Step Trace

To validate our implementation roadmap, we conducted a comparative analysis between the legacy environment (Linux 4.13) and our ported modern environment (Linux 6.16) after implementing the patch. We utilized the step trace to evaluate both kernels. Our first hypothesis claimed that running Orca in its native environment would reproduce the significant latency reduction over TCP Cubic that was reported in the original paper. As predicted, the baseline implementation resulted in an extremely high mean latency (2054.8 ms). The Orca agent was able to reduce the mean latency to 51.5 ms. This benefit came with a minor throughput cost that was consistent with the authors' findings that Orca sacrifices slightly less than optimal throughput to maintain low delay. We successfully reproduced the results of the paper in the original environment.

After porting to a new kernel, we predicted a performance regression, assuming the pre-trained RL model trained on characteristics of the old kernel would struggle with the scheduling and network stack behavior changes made. Contrary to our expectation, the ported kernel actually achieved a higher mean throughput with the Orca implementation, likely because the modern TCP stack or our driver modifications allow the agent to process window updates more efficiently. The delay remained close to identical across both kernels. The port was not only successful but beneficial. The RL model proved robust to underlying OS changes, and the modern kernel environment also provided a more stable execution platform in the legacy system. We also compared baseline Cubic across kernels and found that they performed very similarly. Both exhibited massive delay on the step scenario (∼2000 ms). On Linux 6.16, Cubic performed worse than Orca, isolating the improvement to the Orca implementation itself, rather than a generalized improvement in network emulation.

4.3 Performance Across Algorithms

4.3.1 Sending Rate

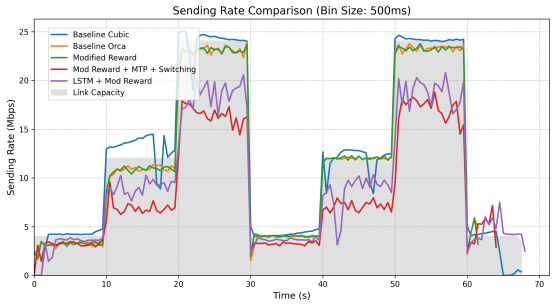

Figure 3: Sending Rate vs. Time: Step Trace

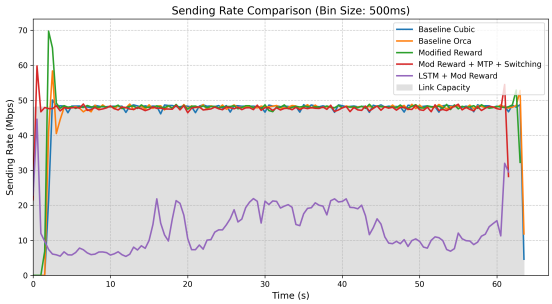

Figure 4: Sending Rate vs. Time: Wired48 Trace

Implementation (avgs over 10 runs)	**Link Util. (%)**	**Windowed Jitter (%)**
Baseline Cubic	101.990	3.219
Baseline Orca (MLP)	94.570	7.113
MLP+Modified Reward	94.680	6.753
MLP+Modified Reward+ Dynamics	93.050	8.719
LSTM+Modified Reward	46.260	9.636

Table 3: Sending Rate Statistics: Step Trace

Implementation (avgs over 10 runs)	**Link Util. (%)**	**Jitter (%)**
Baseline Cubic	99.810	1.964
Baseline Orca (MLP)	97.840	4.319
MLP+Modified Reward	84.420	4.835
MLP+Modified Reward+ Dynamics	99.050	3.033
LSTM+Modified Reward	28.180	17.015

Table 4: Sending Rate Statistics: Wired48 Trace

In this section, we evaluate the sending rate, which is the traffic egress. This is the rate at which packets successfully exit the bottleneck link and reach the receiver. By comparing the sending rate against the link's physical capacity (shaded), we can assess how effectively each algorithm utilizes available bandwidth (whether it is over-provisioning and sending faster than the link can handle, or conservatively under-utilizing the link). Our overall conclusion is that our hypothesis that algorithm switching would prevent performance degradation in most scenarios was more complex. The sending rate graphs show that during bandwidth drops, the drain rate of the queue also slows down, which is identified by the algorithm switching mechanism. However, falling back to an algorithm like Cubic was not necessarily the right choice to fix this high latency, because the RL agent would have backed off to a lower delay.

Step Trace

- **Baseline Cubic**: As expected, the baseline TCP Cubic implementation exposes bufferbloat: Cubic's sending rate consistently exceeds the link capacity, especially during transitions from low to high bandwidth (12 Mbps to 24 Mbps). Cubic is designed to probe for available bandwidth by continuously increasing its window until a packet is dropped, but in this scenario, the 1000 packet buffer absorbs excess traffic without dropping packets. As a result, Cubic does not receive the signal to slow down and continues to ramp its sending rate above the physical limit of the link capacity (26-28 Mbps).

- **Baseline Orca**: The unmodified Orca agent generally tracks the link capacity better than Cubic, but does exhibit high frequency variance. The sending rate curve appears noisy, rapidly oscillating above and below the capacity line. This indicates that while the fixed 20 ms control loop allows for fast reactions, the agent lacks the stability to lock onto the correct bandwidth, constantly over-correcting.

- **Modified Reward**: The modified reward agent acts similarly to Orca because both the baseline Orca and the modified reward agents operate on a fixed 20 ms interval, and this trace has round-trip times of 40 ms or

higher. This forces the agents in both scenarios to make multiple actions before receiving feedback from their first action, causing the oscillation seen in both curves. However, the jitter value shows that the sending rate is more stable with the modified reward.

- **Modified Reward+MTP+Switching**: The results of the hybrid system, with a modified reward, dynamic MTP, and switching, are interesting. During low capacities, it closely tracks the link capacity like Orca and Modified Reward, but at 12 and 24 Mbps, the sending rate does not adapt quickly. This behavior is a deterministic artifact of the dynamic algorithm switching logic. The reduction in link capacity causes an instant spike in RTT and fallback to TCP Cubic. In standard Orca, when falling back to Cubic, the trained agent will add a multiplier to the Cubic congestion window and override its tendency to fill the buffer. The hybrid system disables the DRL agent's modulation entirely. The underutilization is a combination of the RL agent pulling the rate down to fix the delay, and Cubic pushing it up to fill the buffer.

- **LSTM+Modified Reward**: The LSTM model fails to saturate available bandwidth at any capacity level. This underutilization indicates that the agent converged to a conservative local optimum rather than a global maximum. The architectural complexity of the LSTM also introduced interference latency that exceeded the 20 ms control interval. In addition, the link utilization is consistently low, and it has the highest jitter values amongst all the implementations. Because the LSTM model failed to converge during training and hyperparameter tuning was infeasible due to time constraints, a recurring theme in the remaining results will be that this implementation has comparatively poor performance.

Wired48 Trace

All algorithms except for the LSTM successfully converge to a sending rate of approximately 48 Mbps. This confirms that the majority of our modifications did not worsen the agent's performance on a stable network. It also proves that algorithm switching correctly identifies the stable link as a safe environment for the RL agent and does not revert to Cubic or another algorithm unnecessarily. As stated previously, the LSTM fails to reach link capacity due to lack of training.

4.3.2 Throughput vs. Delay

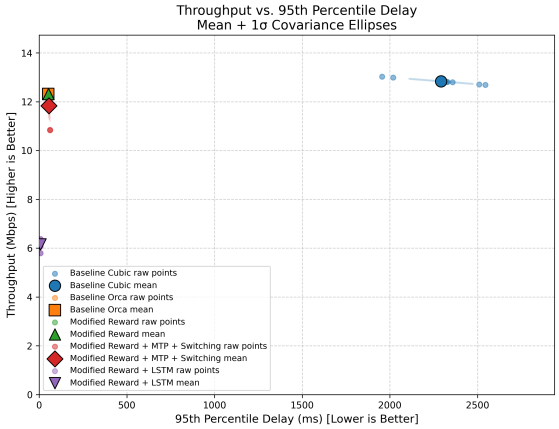

Figure 5: Throughput vs. Delay: Step Trace

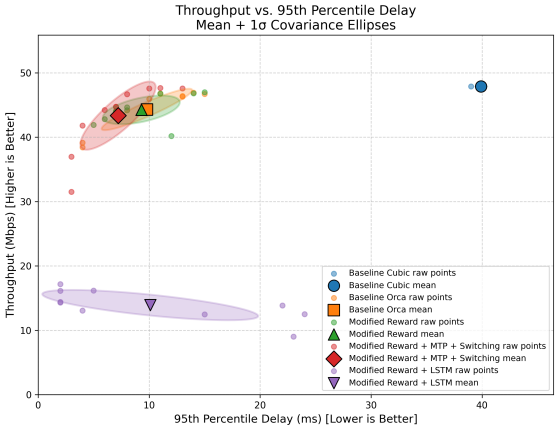

Figure 6: Throughput vs. Delay: Wired48 Trace

Implementation (stats over 10 runs)	Throughput (Mbps)	Delay (ms)
Baseline Cubic	mean: 12.834 var: 0.010	mean: 2292.100 var: 30640.000
Baseline Orca (MLP)	mean: 12.313 var: 0.015	mean: 52.400 var: 20.640
MLP+Modified Reward	mean: 12.321 var: 0.0002	mean: 52.600 var: 1.040
MLP+Modified Reward+ Dynamics	mean: 11.834 var: 0.260	mean: 55.100 var: 16.290
LSTM+Modified Reward	mean: 6.154 var: 0.042	mean: 7.500 var: 0.250

Table 5: Mean and Variance: Step Trace

Implementation (stats over 10 runs)	Throughput (Mbps)	Delay (ms)
Baseline Cubic	mean: 47.881 var: 0.0002	mean: 39.900 var: 0.090
Baseline Orca (MLP)	mean: 44.315 var: 9.353	mean: 9.800 var: 14.960
MLP+Modified Reward	mean: 44.250 var: 4.494	mean: 9.300 var: 10.810
MLP+Modified Reward+ Dynamics	mean: 44.350 var: 25.920	mean: 7.200 var: 10.810
LSTM+Modified Reward	mean: 13.914 var: 4.991	mean: 10.100 var: 85.090

Table 6: Mean and Variance: Wired48 Trace

These graphs show the throughput (Mbps) versus queuing delay (ms). The throughput is the rate at which data is successfully delivered to the receiver. In a bottlenecked link, this is capped by the physical link capacity. The queuing delay is the time the packets spend waiting in the buffer before transmission. A high delay indicates that the sender is pushing data faster than the link can consume it (bufferbloat). We also record variance to examine the stability of the algorithms.

Step Trace

- **Baseline Cubic**: The baseline achieves the highest raw throughput (12.8 Mbps, 99.9% utilization), but has a catastrophic delay of around 2000 ms. The maximum queuing delay is given by

  $$\texttt{Buffer Capacity (Mb)/Link Bandwidth (Mb/s)}$$

  The buffer capacity is 12 Mb ($1000\ \texttt{packets} \times 1.5\texttt{KB/packet} = 12\texttt{Mb}$) and the bottleneck link bandwidth in the step scenario is 4 Mbps. This gives us an approximate worst-case queuing delay of 2 seconds, or 2000 ms. Cubic hits this limit consistently due to bufferbloat consequences described previously. In addition, Cubic has low throughput variance but very high delay variance depending on the exact queue depth at any given moment.

- **Baseline Orca**: Orca resolves the latency issue, reducing the mean delay to 52.4 ms while maintaining a throughput of 12.3. Compared to the Modified Reward agent, the throughput and delay variance are higher but still much better than Cubic. This shows that the original reward function was still able to successfully identify congestion before the buffer fills.

- **Modified Reward**: This is the most stable of all the algorithms. While the mean results are nearly identical to Orca, the variance shows the impact of the log-utility function. The throughput variance dropped by two orders of magnitude, and the delay variance dropped from 20.6 ms to 1.0 ms. By penalizing delay linearly rather than multiplicatively, the gradient of the reward function becomes smoother, and the agent has less oscillation while still maintaining high performance.

- **Modified Reward+MTP+Switching**: The hybrid system has a lower mean throughput and a slightly higher mean delay compared to the Orca agents, and higher variances compared to the Modified Reward agent. The variance is most likely caused by the fact that the kernel prefers the BBR algorithm when there is low delay and loss. As a result, there is more variance than without switching algorithms.

- **LSTM+Modified Reward**: The LSTM model had low throughput, often not able to achieve even 50% of utilization and with a very high jitter value. The reasoning is the same as stated earlier.

Wired48 Trace

- **Baseline Cubic**: The baseline has the highest mean throughput, showing that in a stable environment, Cubic performs exactly as designed. The variance for throughput is also negligible.

- **Baseline Orca**: Orca sacrifices a small amount of bandwidth to achieve a massive reduction in delay, with around 92% utilization, but a mean delay of only 9.8 ms. The DRL agent learned that filling the buffer incurs a delay penalty that outweighs the throughput reward, so it throttles the sending rate slightly below link capacity.
- **Modified Reward**: Like in the step trace, the average performance of the modified reward is practically identical to the baseline Orca. The main differentiator is the throughput variance, which dropped from 9.4 to 4.5. Even in a steady-state scenario, the logarithmic gradient provides smoother feedback than the linear baseline, allowing the agent to maintain the target sending rate with less oscillation.
- **Modified Reward+MTP+Switching**: The hybrid system achieved the lowest mean delay of all functional agents (7.2 ms), but the highest throughput variance (25.9). The reduced delay suggests that the dynamic MTP is working as intended, but the high throughput variance is likely a side effect of the algorithm switching logic. In particular, switching to BBR involves some ramp-up and drain phases, which cause many short term fluctuations, leading to higher throughput variance. In this case, our hypothesis in Section 2 is accurate, since we have a better delay while maintaining about the same throughput and slightly higher variance.
- **LSTM+Modified Reward**: The variance metrics for LSTM are erratic, indicating the agent never found a stable policy, and simply operates with low throughput to avoid delay penalties. This is consistent with all previous results.

4.3.3 CPU Utilization

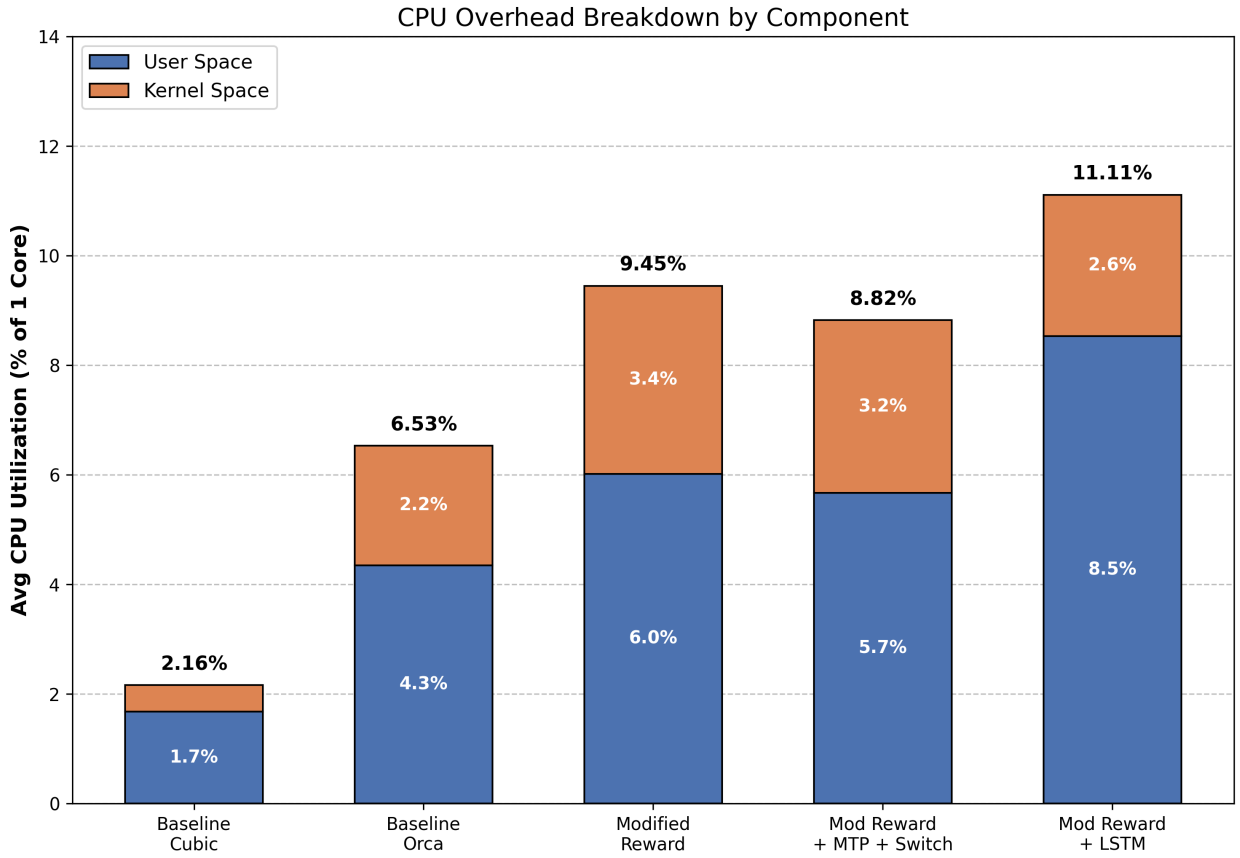

Figure 7: CPU Utilization (Userspace & Kernelspace) Across Algorithms

Figure 7 shows our CPU utilization for the different combinations of ML, algorithm switching, and baselines. CPU utilization was measured by running Orca inside a single-core VM, replaying the workload using a Mahimahi shell, and recording the CPU time of the Orca process with /usr/bin/time, normalized by the wall-clock experiment duration.

- **Baseline Cubic**: This experiment had little overhead since there were no RL decisions that impacted the critical path. This relied solely on the Cubic congestion control algorithm.
- **Baseline Orca**: This experiment had little overhead compared to the other ML experiments since the updates by the agent happened every 20 seconds, and the outputs of the agent were very minimal compared to the other experiments.
- **Modified Reward**: There is a slight increase in userspace and kernelspace percentage compared to the Baseline Orca method, but we believe this is due to experimental variance, because changing the reward function is zero cost and should not have a large impact on CPU utilization.
- **Modified Reward+MTP+Switching**: While this experiment seems like it should've had a larger overhead due to algorithm switching, the kernel would switch algorithms at most every 5 seconds. This was done to ensure that the kernel had enough metadata to confirm the environment that the port was operating in. Furthermore, the MTP was a variable between 20 and 100 seconds, which also limited utilization. By combining these two factors, the overall utilization is lower than the plain Modified Reward.
- **LSTM+Modified Reward**: This agent consumes the highest amount of userspace CPU of any implementation due to the heavy computational load taken by a single LSTM cell (matrix operations and sequential processing of history- compared to MLP, which processes history as a flat snapshot). Because the agent failed to converge and underutilized links, there was a lower kernelspace overhead.

4.4 Cellular Trace

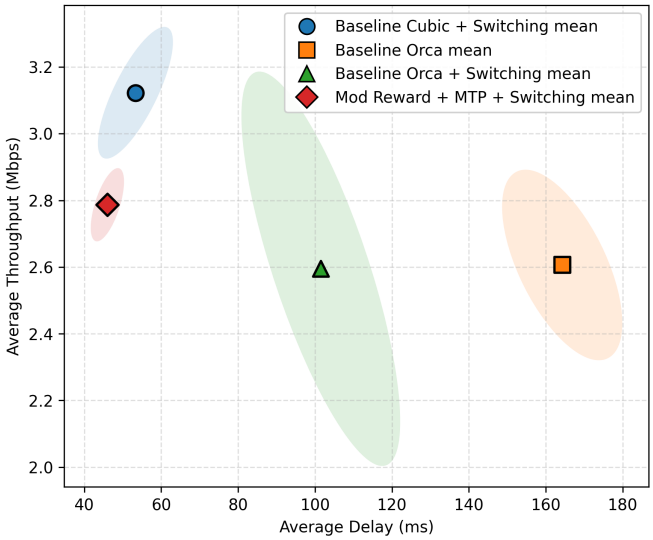

Figure 8: Throughput vs. Delay: Cellular Trace

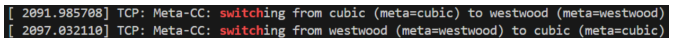

Figure 9: Successfully Switching Congestion Control Algorithm

Cellular Trace

- **Baseline Cubic+Switching**: Cubic is not tailored for Cellular conditions, so properly identifying and switching to a better algorithm, such as Westwood, as seen in Figure 9, leads to average throughput increasing by .5 Mbps and average delay decreases by 110 ms.

- **Baseline Orca**: Orca relies on the Cubic congestion control algorithm, so it will perform quite poorly when placed in conditions that are very different from the ones it was trained on, with a baseline algorithm that is also not optimal for cellular traces.
- **Baseline Orca+Switching**: When the baseline algorithm that Orca enhances is suitable for cellular traces, the average delay improves; however, the variance drastically increases. This is expected since the RL agent was not trained to fall back on Westwood, the selected algorithm for cellular traces.
- **Modified Reward+MTP+Switching**: This result has some improved features compared to other combinations of agent with algorithm switching. First, is the small variance. This is a result of the agent being more in sync with the proper data as discussed in Section 2. Next, this configuration also achieved the lowest average delay among all implementations. By switching to Westwood, the system handles the stochastic packet loss inherent in the driving trace without interpreting it as congestion.

5 Conclusion

5.1 Roadblocks

Our first set of challenges was purely related to understanding the environment. Although Mahimahi is widely used in congestion-control research, neither of us had prior experience with it. We had to understand how its basic commands worked, how it interacted with Linux, and how to interpret the result logs. The second major roadblock was reconstructing the original Orca software environment. The reference implementation targets a custom 4.13 Linux kernel patch and TensorFlow 1.14 running inside a Python 3.7 virtual environment. TensorFlow 1.14 is now deprecated and not supported in recent distributions, and several of its APIs do not have drop-in replacements in modern TensorFlow versions. As a result, we could not simply "modernize" the code by upgrading the ML stack.

Matching the exact experimental conditions was also difficult. While the public repository has some scripts and configuration files, some of the parameters used to obtain the data, such as queue size, have to be inferred from comments, default parameters, and the authors' description. We were able to reconstruct experiments that resemble the published results, but small discrepancies in trace selection or queue sizing can shift points in the figures.

For the extension of switching congestion-control algorithms in the kernel, a challenge was understanding the Linux code, as it's the first time we've changed Linux kernel code. Once we figured out where we could place our dynamic algorithm selection, it became easier to experiment with different heuristics for algorithm switching.

On the machine learning front, our primary challenge was architectural complexity versus training efficiency. Attempting to replace Orca's simple MLP with an LSTM network to better capture temporal network states incurred severe overhead because the LSTM inference time often exceeded the 20 ms control interval (causing the C++ server to timeout and stalling the training process for hours), and unlike the MLP, the LSTM agent failed to converge to a performant policy within 500000 steps. Given that the prohibitive training latency made iterative hyperparameter tuning infeasible within our project timeline, we ultimately reverted to the initial MLP architecture for all further system extensions.

5.2 Improvements & Future Work

Based on our findings, we propose several key improvements to the current system and directions for future research:

1. Containerizing the entire stack such that reproducing results becomes easier for new users, rather than manually installing legacy dependencies.
2. We currently use a single DRL agent, but because different networks vary wildly in latency and capacity, it may be useful to train specialized DRL agents paired with specific underlying algorithms, and dynamically switch not just the base algorithm but also the controlling agent.
3. Currently, Orca relies on a legacy architecture involving block ing `setsockopt` calls for communication. This IPC overheaf becomes a bottleneck when using more complex models (LSTM) due to high inference time. In the future, we would like to replace the patch with eBPF to remove the need for a custom kernel and enabling

non-blocking data exchange via BPF maps. Fully embedding the neural network inference within the kernel is not feasible, but optimizing the bridge between kernel and userspace would reduce the latency that hinders the training of temporally-aware models.

5.3 Miscellaneous

5.3.1 How Much Time Spent?

Around 7 weeks.

5.3.2 Who Did What?

- What we did together: Reproducing Orca on the 4.13 kernel and implementing the patch on the 6.16 kernel, general learning about how different parts of the architecture worked with each other, and the report.
- What Krithi did: Adding RL modifications, dynamic MTP, support for algorithm switching in userspace, and final data collection on step and wired48 traces.
- What Mircea did: Adding dynamic algorithm switching in kernelspace, final data collection 4.13 kernel and cellular traces.

5.3.3 Repositories

Both repositories are shared with cs380l-aos-ta.

- The modified Linux kernel can be found here: https://github.com/MirceaNT/OrcaLinux/tree/CCA_Change_Attempt1
- The modified Orca (userspace) repository can be found here: https://github.com/krisub/OrcaRL

5.3.4 Report Modifications

Flowcharts related to ML models can be found in the presentation slides (see documentation) since they are not systems related. These are the report modifications we made after receiving presentation feedback:

- Removed scientific notation from tables.
- Explained kernel patch with slightly more detail.
- Looked into how we can discover from the kernel what the current network type is.
- Added measurements on area under the curve (link utilization) and oscillation amplitude (jitter) for sending rate graphs.
- Changed tables and figure in appendix so they are more readable. This also includes removing anything related to data centers since we didn't evaluate any traces that properly emulate data centers.

References

Ravinet. [n. d.]. Mahimahi: Web performance measurement toolkit. `https://github.com/ravinet/mahimahi`. Accessed: 2025-12-01.

Soheil-Ab. [n. d.]. Orca: Towards Mastering Congestion Control In the Internet. `https://github.com/Soheil-ab/Orca`. Accessed: 2025-12-01.

H. Jonathan Chao Soheil Abbasloo, Chen-Yu Yen. 2020. Classic Meets Modern: A Pragmatic Learning-Based Congestion Control for the Internet. In *ACM SIGCOMM*. doi:`10.1145/3387514.3405892`

A Appendix

Table 8: Meta-CC per-flow state (`struct tcp_meta_state`); total per-socket memory overhead is **56 bytes**. Total socket size is **2432 bytes**

Field	**Type**	**Purpose**
`enabled`	u8	Flag indicating whether the meta-CC logic is enabled and managing this flow.
`current_algo`	enum	Identifies which underlying congestion control algorithm is currently active for the flow.
`flags`	u8	Reserved flag bits for future meta-CC hints (e.g. latency sensitivity or application requirements).
`pad`	u8	Unused padding to keep subsequent 32-bit fields properly aligned.
`badness`	u32	Counter tracking persistent "bad" conditions (for example, sustained high queueing delay) used to trigger CC switching.
`base_rtt_us`	u32	Minimum RTT observed (in µs), used as baseline propagation delay for computing queueing delay.
`qdelay_ewma_us`	u32	EWMA of queueing delay (in µs).
`rtt_var_us`	u32	EWMA of RTT variability (in µs), used as a jitter/variability signal.
`ecn_ewma`	u32	EWMA of ECN-marked packets in permille (0–1000), representing ECN-based congestion level on the path.
`loss_ewma`	u32	EWMA of packet loss or retransmission rate in permille (0–1000), representing loss-based congestion level.
`last_delivered`	u32	Snapshot of `tp->delivered` at the last meta-CC sampling point, to measure delivered-bytes deltas.
`last_delivered_ce`	u32	Snapshot of `tp->delivered_ce` at the last sampling point, to track new ECN-marked delivered packets.
`last_total_retrans`	u32	Snapshot of `tp->total_retrans` at the last sampling point, to detect additional retransmissions since then.
`last_switch`	ul	Timestamp of the most recent CC algorithm switch, used to rate-limit meta-CC transitions over time.

Table 9: TCP meta environments and classification characteristics

Environment	**Classification rule**
`UNKNOWN`	No RTT sample yet, so the environment is not classified.
`DATACENTER_ECN`	very short RTT and a noticeable ECN-marking ratio (datacenter with ECN).
`WIRELESS`	high loss but queueing delay not large (wireless / random-loss).
`HIGH_BDP`	large base RTT with low loss, indicating a high-BDP long fat network.
`INTERNET`	Any case with $base\_rtt \neq 0$ that does not satisfy the rules above is treated as generic Internet.

Kernel option	Setting	What it does / why Orca needs it
NF_CONNTRACK	y (via --enable)	Enables the core Netfilter connection tracking subsystem. The kernel keeps per-flow state (ct entries) and is able to tell which packets belong to which connection; required for any iptables rules that use connection state or connection marks, and for NAT-based emulation.
NF_CONNTRACK_MARK	y (via --enable)	Adds support for a per-connection "mark" field stored in the conntrack entry (ctmark). This is what the CONNMARK target and connmark match operate on, letting the emulation script tag flows once and then classify all packets of that flow by the stored mark.
NETFILTER_XT_TARGET_CONNMARK	m (via --module)	Builds the xt_CONNMARK target as a module. This target copies marks between packet mark and connection mark (e.g., -j CONNMARK --save-mark / --restore-mark), which is typically used by the emulator's iptables rules to propagate decisions between individual packets and their conntrack state.
NETFILTER_XT_MATCH_CONNMARK	m (via --module)	Builds the xt_connmark match as a module. This allows iptables rules like -m connmark --mark 0xNN/0xFF to match packets based on the connection's ctmark, which Orca's emulation script can use to select different paths, queues, or shaping policies per flow.
TUN	y (via --enable)	Enables the universal TUN/TAP driver as a built-in. The script can create virtual TUN/TAP interfaces to route traffic through the emulator; packets are injected to / read from user space instead of a physical NIC, which is how the standalone emulation environment gets its "virtual link".
NETFILTER_ADVANCED	y (via --enable)	Turns on the "advanced" Netfilter/Xtables options menu. Many of the more specialized matches/targets (including connmark/CONNMARK) depend on this being enabled, so it is a prerequisite for the connmark-based rules used by the Orca setup.
MODULE_SIG	n (via --disable)	Disables kernel module signature verification. With this turned off, the kernel will load unsigned or locally-built modules without requiring them to be cryptographically signed, which avoids "module verification failed" errors when you experiment with custom or out-of-tree modules in the Orca kernel.
MODULE_SIG_ALL	n (via --disable)	Disables the policy that would otherwise require *all* modules to carry a valid signature. This keeps module loading permissive, so development / debug builds of the kernel and any supporting modules for Orca can be inserted without going through a signing toolchain.

Table 7: Kernel configuration options enabled/disabled to run Orca experiments.
Note: MODULE_SIG and MODULE_SIG_ALL shouldn't need to be disabled to run Orca experiments. We disabled them since Orca wasn't running and we thought that was the problem. After discovering the real issue, we never re-enabled the kernel modules

Table 10: Congestion control algorithms used by the meta selection logic

Algorithm	**Ideal / intended environment**	**Key signals / parameters**
`westwood`	Wireless or random-loss paths with variable capacity.	ACK-rate-based bandwidth estimate; loss events rescale congestion window and pacing rate.
`bbr`	High-BDP, relatively low-loss paths; also good for low-latency generic Internet.	Bottleneck bandwidth from delivery rate; minimum RTT; drives pacing rate and cwnd toward BDP.
`illinois`	High-speed, high-BDP paths with non-trivial queueing delay.	Packet loss plus queueing delay (RTT−base RTT) to adapt cwnd growth and reduction.
`cubic`	Generic Internet, especially long-fat paths without ECN.	Packet loss as primary congestion signal; cubic cwnd growth as a function of time since last loss.

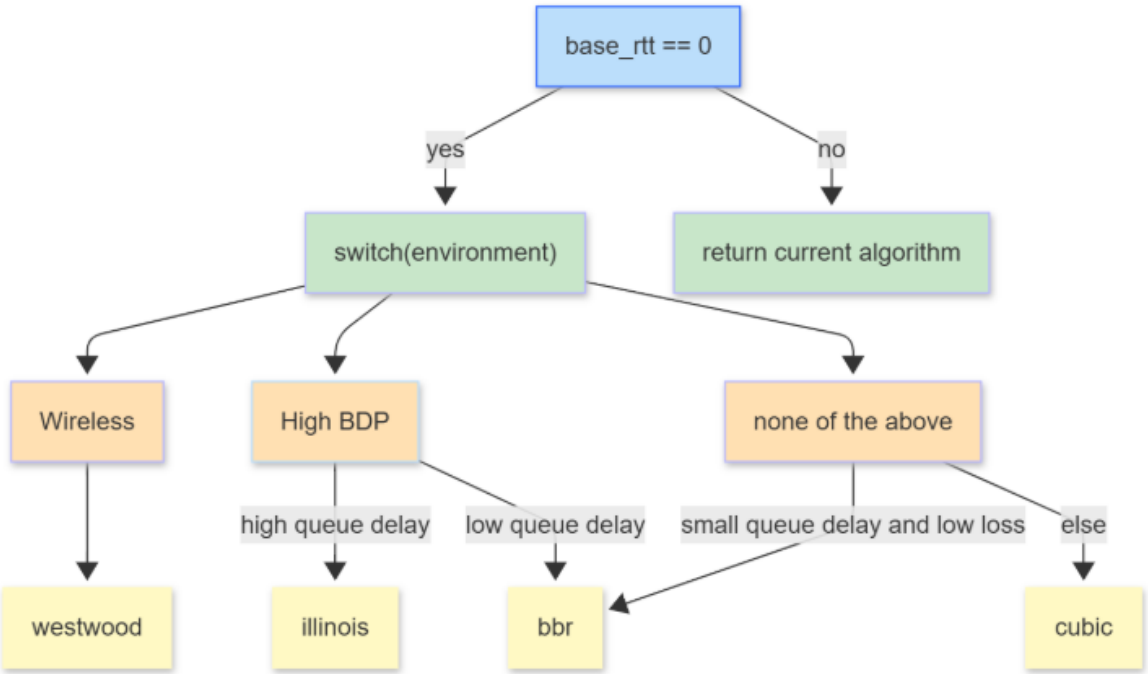

Figure 10: Flow chart of the TCP meta-CC algorithm selection logic.